

FIAP

Inteligência Artificial

RELATÓRIO DE ATIVIDADE – FASE 3

Monitoramento Contínuo – IoT na Saúde

Projeto CardioloA Conectada

Integrantes:

Nome Completo	RM
Daniele Antonieta Garisto Dias	RM565106
Leandro Augusto Jardim da Cunha	RM561395
Luiz Eduardo da Silva	RM561701
João Victor Viana de Sousa	RM565136

São Paulo, 2026

Sumário

Sumário.....	2
PARTE 1 – Armazenamento e Processamento Local (Edge Computing).....	3
1.1 Visão Geral.....	3
1.2 Sensores Utilizados.....	3
Sensor 1 – DHT22 (Temperatura e Umidade).....	3
Sensor 2 – PIR (Sensor de Movimento).....	3
1.3 Fluxo de Funcionamento.....	3
1.4 Lógica de Resiliência Offline (Edge Computing).....	3
Alternativa para SPIFFS.....	4
1.5 Simulação de Conectividade.....	4
1.6 Alertas Locais (Regras de Negócio).....	4
PARTE 2 – Transmissão para Nuvem e Visualização (Fog/Cloud Computing)	5
2.1 Visão Geral.....	5
2.2 Sensores Utilizados.....	5
Sensor 1 – DHT22 (Temperatura).....	5
Sensor 2 – Botão (Simulação de Batimentos Cardíacos).....	5
2.3 Protocolo MQTT e Configuração do Broker.....	6
2.4 Fluxo Completo de Comunicação.....	7
Camada Edge – ESP32.....	7
Camada Fog – Node-RED.....	7
Camada Cloud – InfluxDB + Grafana.....	7
2.5 Dashboard Node-RED.....	8
2.6 Integração com Grafana e InfluxDB.....	10
3. Arquitetura Geral do Sistema.....	12
4. Limitações e Trabalhos Futuros.....	13
4.1 Limitações Identificadas.....	13
4.2 Trabalhos Futuros.....	13
5. Considerações Finais.....	15
Referências.....	16

PARTE 1 – Armazenamento e Processamento Local (Edge Computing)

1.1 Visão Geral

A Parte 1 consiste em uma aplicação desenvolvida no simulador Wokwi com o microcontrolador ESP32. O objetivo é demonstrar os princípios de Edge Computing aplicados à saúde digital: o dispositivo realiza coleta, processamento local e gerenciamento de resiliência offline de dados de sinais vitais, sem depender de conexão permanente com a nuvem.

1.2 Sensores Utilizados

Sensor 1 – DHT22 (Temperatura e Umidade)

Conectado ao pino GPIO 15, o DHT22 é responsável pela leitura periódica de temperatura (graus Celsius) e umidade relativa do ar (%). Por se tratar de um sensor dual, é contabilizado como um único sensor conforme especificação do enunciado. As leituras são validadas com a função `isnan()` para garantir a integridade dos dados antes de qualquer processamento.

Sensor 2 – PIR (Sensor de Movimento)

Conectado ao pino GPIO 4, o sensor PIR detecta presença no ambiente. No contexto do CardiolA, representa a detecção de movimento do paciente monitorado, sendo parâmetro relevante em cenários de queda ou imobilidade prolongada - situações críticas em pacientes cardiológicos.

1.3 Fluxo de Funcionamento

O sistema opera em ciclos de leitura a cada 5 segundos, controlados por temporização não bloqueante (`millis()`). A cada ciclo:

- O ESP32 lê temperatura, umidade e estado do sensor PIR.
- Os dados são serializados em formato JSON usando a biblioteca `ArduinoJson`.
- O sistema verifica o estado de conectividade (variável booleana `isOnline`).
- Dependendo do estado de conexão, os dados são enviados ou armazenados localmente.

O payload gerado segue o formato JSON abaixo:

```
{ "temperatura": 36.5, "umidade": 62.0, "movimento": false }
```

1.4 Lógica de Resiliência Offline (Edge Computing)

A resiliência offline é implementada por meio de um buffer local em memória RAM do ESP32, com capacidade para armazenar até 20 leituras - estratégia definida com base no modelo de negócio do projeto.

Funcionamento do Buffer

- Quando o sistema está OFFLINE: cada nova leitura é salva no array `buffer[]` por meio da função `saveLocal()`.
- Quando o buffer atinge a capacidade máxima, aplica-se política FIFO (First In, First Out): a leitura mais antiga é descartada e a mais recente ocupa o último slot.
- Quando a conexão é RESTABELECIDA: a função `syncData()` percorre todos os registros, exibe-os no Monitor Serial simulando o envio para a nuvem, e limpa o buffer.

Justificativa de capacidade: 20 amostras x 5 segundos = 100 segundos (~1,6 min) de operação offline sem perda de dados. Para um dispositivo vestível cardíaco hospitalar, essa janela cobre perdas de sinal momentâneas em ambientes com interferência eletromagnética ou transição entre alas.

Alternativa para SPIFFS

Como o simulador Wokwi não suporta persistência SPIFFS (o sistema de arquivos é volátil ao reiniciar a simulação), o Monitor Serial foi adotado como alternativa de saída, conforme orientação do enunciado. Em um chip ESP32 físico, os dados poderiam ser gravados em arquivo CSV via SPIFFS ou em cartão microSD.

1.5 Simulação de Conectividade

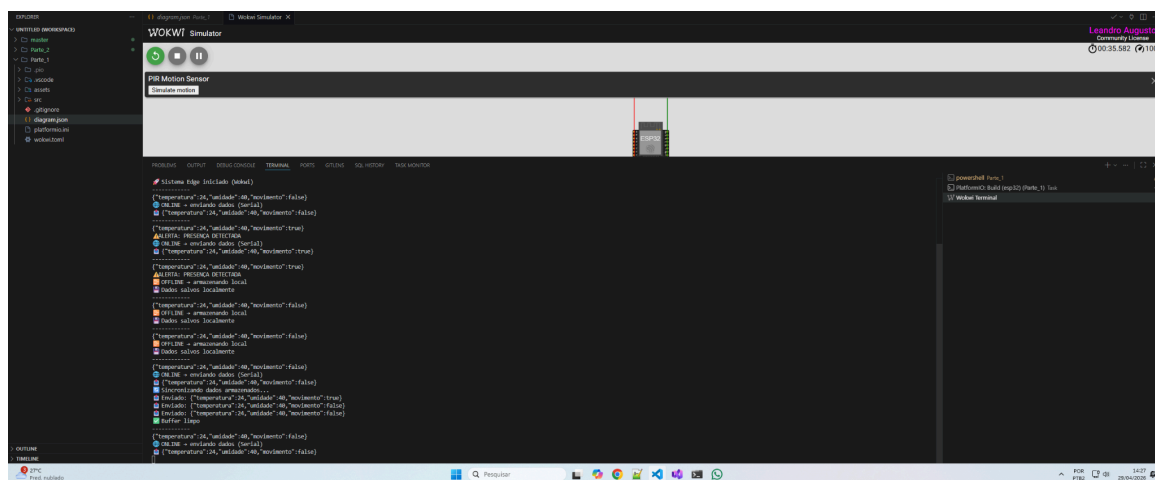
A conectividade é simulada automaticamente pela própria lógica do `loop()`: a variável `isOnline` alterna entre `true` e `false` a cada 15 segundos dentro de um ciclo de 30 segundos usando `millis() % 30000`. Isso permite observar no Monitor Serial o comportamento completo do sistema tanto em modo online quanto offline, sem necessidade de intervenção manual.

1.6 Alertas Locais (Regras de Negócio)

Além do armazenamento, o sistema implementa regras de alerta processadas localmente no ESP32:

- Temperatura > 38°C: exibe “ALERTA: FEBRE DETECTADA” no Monitor Serial.
- Movimento detectado pelo PIR: exibe “ALERTA: PRESENÇA DETECTADA”.

Esses alertas demonstram que o ESP32, como dispositivo de borda, executa lógica de negócio crítica sem depender de conexão com a nuvem - característica fundamental em sistemas médicos que exigem resposta imediata.



PARTE 2 – Transmissão para Nuvem e Visualização (Fog/Cloud Computing)

2.1 Visão Geral

A Parte 2 integra o ESP32 a uma infraestrutura completa de Fog e Cloud Computing, composta por broker MQTT (Mosquitto local), Node-RED como camada de fog, InfluxDB como banco de dados de séries temporais e Grafana para visualização avançada. O objetivo é demonstrar o fluxo completo de dados desde a captura no dispositivo até a exibição em dashboards com alertas automáticos.

2.2 Sensores Utilizados

Sensor 1 – DHT22 (Temperatura)

Conectado ao GPIO 15, realiza leitura de temperatura a cada 10 segundos. Os dados são publicados no tópico MQTT sensor/temperatura no formato JSON:

```
{ "temperatura": 36.5 }
```

Sensor 2 – Botão (Simulação de Batimentos Cardíacos)

Conectado ao GPIO 22 com INPUT_PULLUP, o botão simula os batimentos cardíacos do paciente. O usuário pressiona o botão repetidamente para representar batimentos, e o sistema conta os acionamentos ao longo de uma janela de 10 segundos. O cálculo de BPM é feito pela fórmula:

$$\text{BPM} = \text{contagem_de_pressoes} \times 6$$

Um LED no GPIO 17 fornece feedback visual a cada batimento detectado. Os dados são publicados no tópico sensor/batimentos no formato:

```
{ "bpm": 72 }
```

2.3 Protocolo MQTT e Configuração do Broker

MQTT (Message Queuing Telemetry Transport) é um protocolo de mensageria leve baseado no padrão publish/subscribe, ideal para dispositivos IoT com recursos limitados. Suas principais características para este projeto são:

- Baixo overhead: headers pequenos, adequado para ESP32 com memória limitada.
- Desacoplamento: o ESP32 publica dados sem precisar saber quem os consome.
- QoS configurável: garante entrega confiável mesmo em redes instáveis.
- Reconexão automática: função `reconnect_mqtt()` restabelece conexão em caso de queda.

Configuração utilizada no projeto:

Parametro	Valor
Broker	Mosquitto (local) - 192.168.0.120:1883
Client ID	esp32-fiap-01
Tópico 1	sensor/temperatura
Tópico 2	sensor/batimentos
Biblioteca	PubSubClient (Arduino)

```

28 Apr 18:08:08 - [Info] Stopped flows
28 Apr 18:08:08 - [Info] Updated flows
28 Apr 18:08:08 - [Info] Starting flows
TypeError: Cannot read properties of null (reading 'config')
    at new LfNodeConstructor (/home/leonid@linux: node-red/node_modules/node-red-contrib-ui-led/dist/nodes/ui_led.js:25:44)
    at Object.createNode (/usr/local/lib/node_modules/node-red/node_modules/node-red-contrib-ui-led/dist/nodes/ui_led.js:211:27)
    at flow.start (/usr/local/lib/node_modules/node-red/node_modules/node-red-contrib-ui-led/dist/nodes/ui_led.js:264:58)
    at async.start (/usr/local/lib/node_modules/node-red/node_modules/node-red-contrib-ui-led/dist/nodes/ui_led.js:403:17)
28 Apr 18:08:08 - [Info] Started flows
28 Apr 18:08:08 - [Info] [mqtt-broker:MQSquito-Mqtt-Temperature] Connected to broker: mqtt://192.168.0.120:1883
28 Apr 18:08:08 - [Info] [mqtt-broker:MQSquito-Mqtt-Spn] Connected to broker: mqtt://192.168.0.120:1883
28 Apr 18:09:36 - [Info] Stopping flows
28 Apr 18:09:36 - [Info] [mqtt-broker:MQSquito-Mqtt-Temperature] Disconnected from broker: mqtt://192.168.0.120:1883
28 Apr 18:09:36 - [Info] [mqtt-broker:MQSquito-Mqtt-Spn] Disconnected from broker: mqtt://192.168.0.120:1883
28 Apr 18:09:36 - [Info] Stopped flows
28 Apr 18:09:36 - [Info] Updated flows
28 Apr 18:09:36 - [Info] Starting flows
TypeError: Cannot read properties of null (reading 'config')
    at new LfNodeConstructor (/home/leonid@linux: node-red/node_modules/node-red-contrib-ui-led/dist/nodes/ui_led.js:25:44)
    at Object.createNode (/usr/local/lib/node_modules/node-red/node_modules/node-red-contrib-ui-led/dist/nodes/ui_led.js:211:27)
    at flow.start (/usr/local/lib/node_modules/node-red/node_modules/node-red-contrib-ui-led/dist/nodes/ui_led.js:264:58)
    at async.start (/usr/local/lib/node_modules/node-red/node_modules/node-red-contrib-ui-led/dist/nodes/ui_led.js:403:17)
28 Apr 18:09:36 - [Info] Started flows
28 Apr 18:09:36 - [Info] [mqtt-broker:MQSquito-Mqtt-Temperature] Connected to broker: mqtt://192.168.0.120:1883
28 Apr 18:09:36 - [Info] [mqtt-broker:MQSquito-Mqtt-Spn] Connected to broker: mqtt://192.168.0.120:1883
28 Apr 18:10:07 - [Info] [debug:Recebidu Temperatura]
{
  topic: 'sensor/temperatura',
  payload: { temperatura: 36.3, umidade: 40 },
  qos: 0,
  retain: false,
  _msgid: '2b639bdc4e6b6d68'
}
28 Apr 18:10:07 - [Info] [debug:Recebidu Batimento]
{
  topic: 'sensor/batimentos',
  payload: { bpm: 0 },
  qos: 0,
  retain: false,
  _msgid: 'c9719c4088a760bc'
}
28 Apr 18:10:18 - [Info] [debug:Recebidu Temperatura]
{
  topic: 'sensor/temperatura',
  payload: { temperatura: 36.3, umidade: 40 },
  qos: 0,
  retain: false,
  _msgid: '550b69d00606a3c5'
}
28 Apr 18:10:18 - [Info] [debug:Recebidu Batimento]

```

2.4 Fluxo Completo de Comunicação

O fluxo de dados segue a arquitetura em três camadas:

```
ESP32 --> [MQTT] --> Mosquitto Broker --> Node-RED --> InfluxDB --> Grafana
```

Camada Edge – ESP32

- Coleta temperatura via DHT22 a cada 10 segundos.
- Conta batimentos via botão e calcula BPM a cada 10 segundos.
- Serializa os dados em JSON e pública via MQTT nos tópicos configurados.
- Mantém reconexão automática ao broker (resiliência na transmissão).

Camada Fog – Node-RED

- Subscrive os tópicos MQTT sensor/temperatura e sensor/batimentos.
- Realiza parte do JSON recebido e estrutura os dados.
- Aplica regras de alerta: temperatura > 38°C dispara alerta de febre; BPM > 120 dispara alerta de taquicardia.
- Exibe dados em tempo real nos dashboards (gráficos de linha, medidores gauge e alertas visuais).
- Encaminha os dados para o InfluxDB para armazenamento histórico.

Camada Cloud – InfluxDB + Grafana

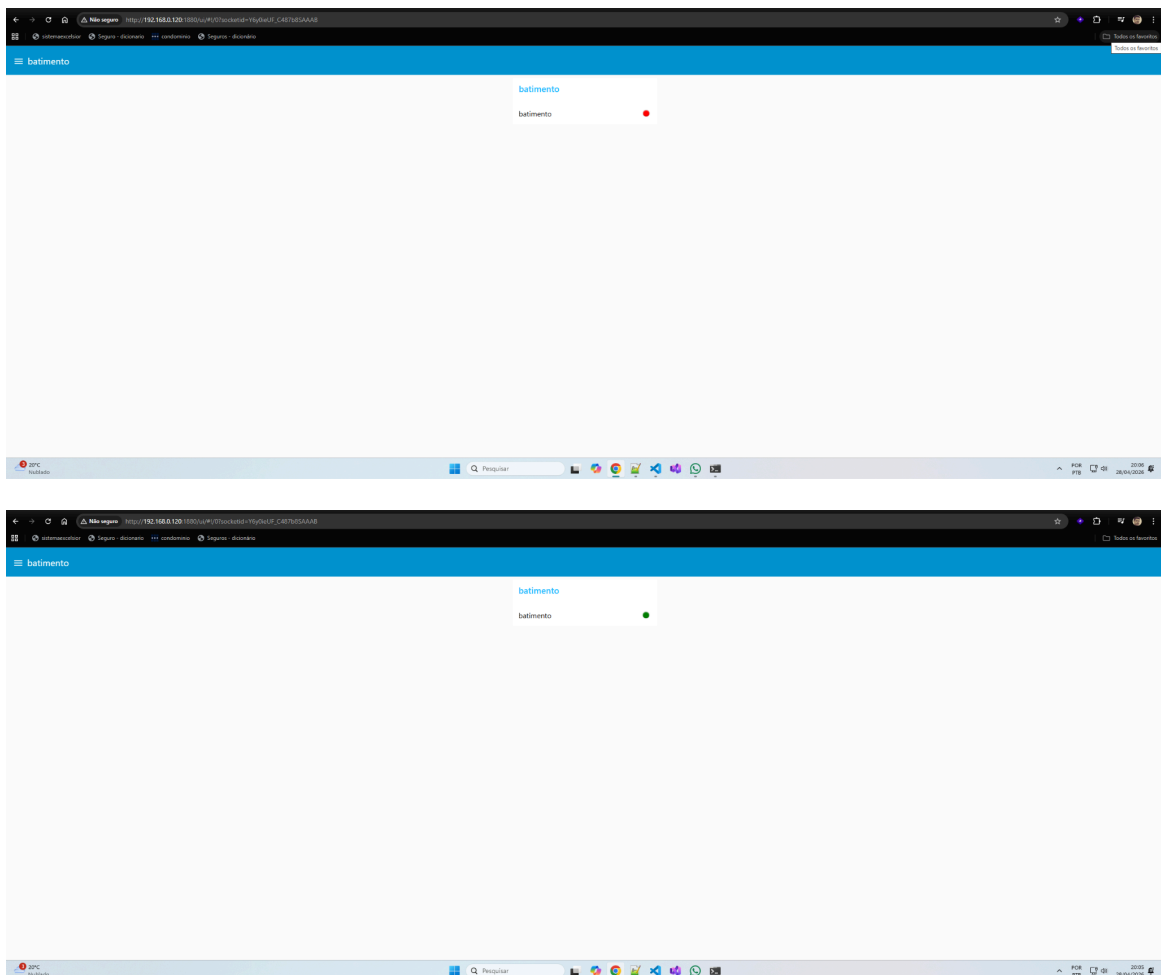
- O InfluxDB armazena os dados como séries temporais com timestamps automáticos.
- O Grafana consome os dados do InfluxDB e exibe dashboards interativos.
- Dashboards incluem: gráfico histórico de temperatura, gráfico de BPM, indicadores gauge e painéis de alerta.

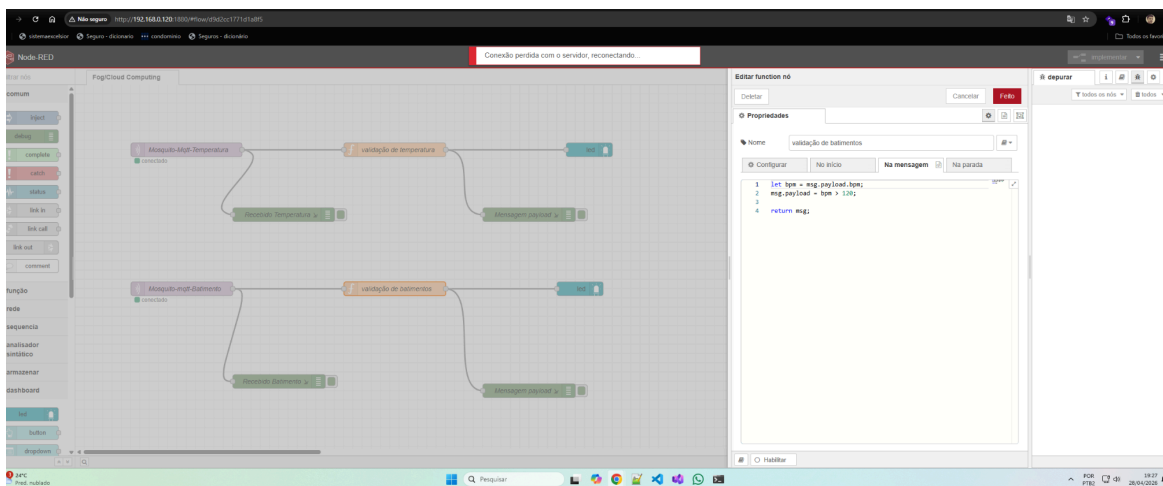
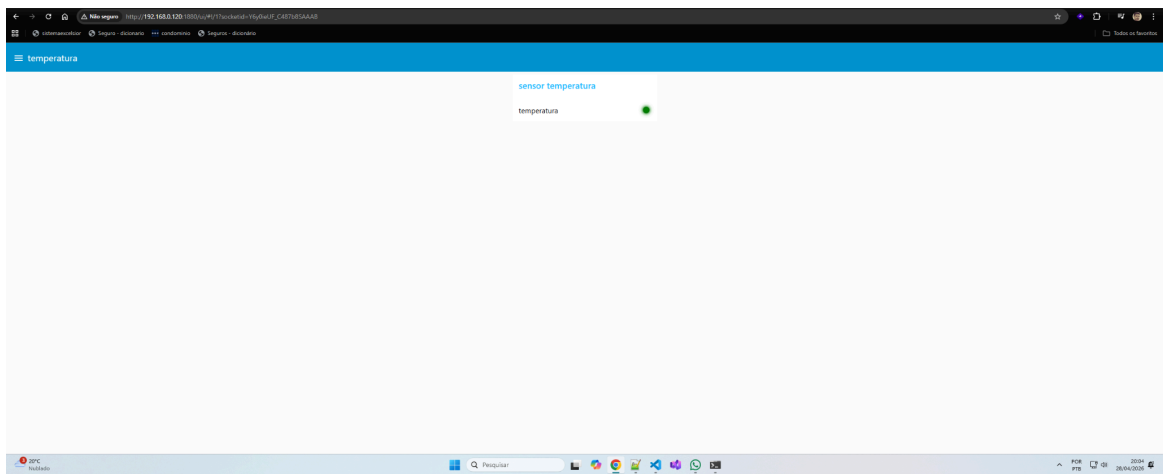
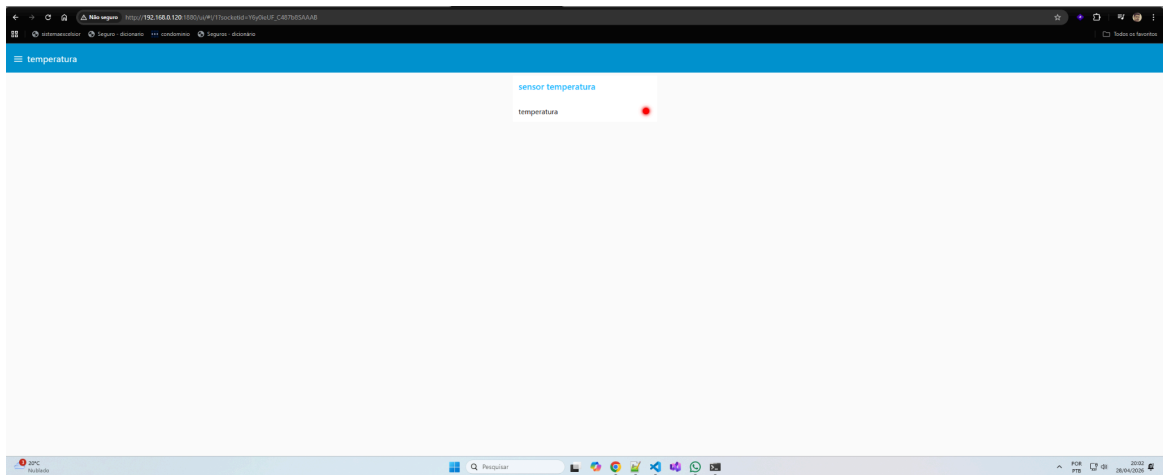
2.5 Dashboard Node-RED

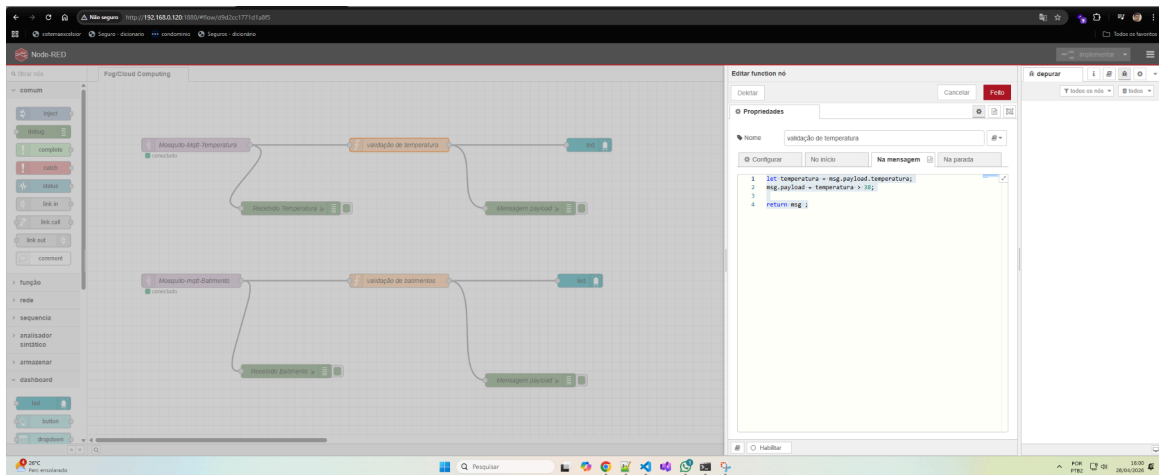
O dashboard foi configurado com os seguintes componentes visuais:

- Gráfico de linha (Chart): exibe a variação dos batimentos cardíacos (BPM) ao longo do tempo em tempo real.
- Medidor (Gauge): exibe a temperatura atual com escala visual de 0 a 50°C.
- Indicador de alerta: muda de cor e exibe mensagem quando temperatura > 38°C ou BPM > 120.

As evidências do funcionamento do dashboard estão registradas nas capturas de tela disponibilizadas na pasta assets/ do repositório do projeto: *dashboard_batimento_alert.png*, *dashboard_batimento_sucess.png*, *dashboard_temperatura_alert.png* e *dashboard_temperatura_sucess.png*.



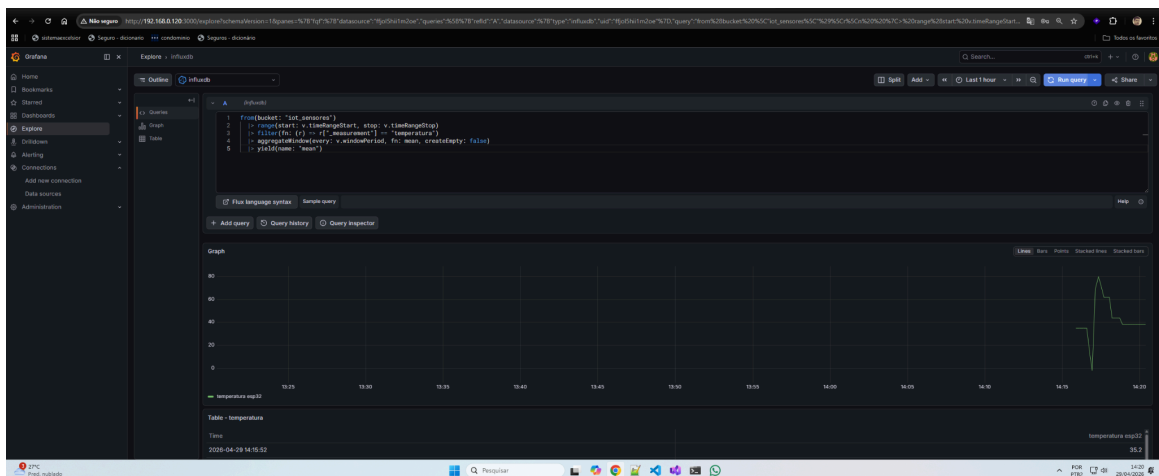




2.6 Integração com Grafana e InfluxDB

Além do Node-RED, o projeto integra o Grafana com o InfluxDB para visualização avançada dos dados históricos. Essa integração permite consultas com linguagem Flux para análise de tendências, criação de painéis customizados com múltiplos gráficos e alertas persistentes, e armazenamento de longo prazo dos sinais vitais coletados.

As capturas de tela das interfaces estão registradas em *grafana.png*, *influxdb.png* e *imagem_node_red.png* na pasta *assets/* do repositório.



22°C Pred: nublado POR 14:00 28/04/2025



3. Arquitetura Geral do Sistema

O CardiolA Fase 3 implementa uma arquitetura IoT distribuída em três camadas bem definidas, conforme descrito na tabela a seguir:

Camada	Tecnologia	Funcao
Edge	ESP32 + DHT22 + PIR/Botao	Coleta, processamento local, resiliência offline
Fog	Node-RED + Mosquitto	Roteamento, regras de negócio, dashboard em tempo real
Cloud	InfluxDB + Grafana	Armazenamento histórico, análise e visualização avançada

Essa divisão de responsabilidades garante:

- Baixa latência: alertas críticos são processados localmente no ESP32 sem depender de rede.
- Escalabilidade: novos sensores podem ser adicionados sem alterar a camada de nuvem.
- Resiliência: o sistema continua operando mesmo sem conexão (buffer FIFO).
- Rastreabilidade: todos os dados ficam persistidos com timestamp no InfluxDB.
- Observabilidade: dashboards em tempo real permitem monitoramento contínuo do paciente.

4. Limitações e Trabalhos Futuros

4.1 Limitações Identificadas

O presente trabalho apresenta limitações inerentes ao escopo de simulação e às escolhas de design adotadas. Reconhecê-las é fundamental para situar os resultados no contexto acadêmico e orientar as evoluções necessárias antes de qualquer aplicação clínica real:

- Ambiente simulado (Wokwi): ausência de persistência SPIFFS, sem conectividade Wi-Fi real e sem latência de rede. O comportamento do firmware em hardware físico pode divergir em aspectos de temporização e confiabilidade.
- Sensor de temperatura ambiente: o DHT22 mede a temperatura do ar, não corporal. Substituição por sensor de contato é necessária para aplicação clínica real.
- Simulação de BPM por botão: não captura variabilidade de frequência cardíaca (HRV) nem eventos arrítmicos como fibrilão atrial ou extras-sístoles.
- Segurança da comunicação: a comunicação MQTT não utiliza TLS/mTLS nem autenticação no broker. Em ambiente de produção com dados de saúde, isso seria inaceitável do ponto de vista regulatório (LGPD, HIPAA).
- Capacidade de buffer limitada: 100 segundos de cobertura offline são insuficientes para falhas de rede prolongadas em ambiente hospitalar real.
- Limiadores de alerta fixos: os valores de 38°C e 120 BPM são parâmetros genéricos; em produção, deveriam ser configuráveis por perfil de paciente.

4.2 Trabalhos Futuros

Com base nas limitações identificadas, propõem-se as seguintes evoluções para fases subsequentes do projeto:

- Integração com hardware físico: validação do firmware em ESP32 DevKit com sensores reais e rede Wi-Fi de produção.
- Sensor de ECG real: substituição do botão pelo módulo AD8232, permitindo análise de intervalo R-R e detecção de arritmias.
- Segurança end-to-end: implementação de TLS no broker Mosquitto, autenticação por certificado cliente e criptografia dos dados em repouso no InfluxDB.
- Buffer persistente: armazenamento em SPIFFS ou cartão microSD para suportar horas de operação offline.

- Parametrização clínica: painel de configuração de limiares de alerta por paciente, com integração a prontuário eletrônico.
- Notificações push: integração do Node-RED com serviço de mensageria (Telegram Bot, PagerDuty) para alertas fora do dashboard.

5. Considerações Finais

O projeto demonstrou de forma prática o ciclo completo de um sistema IoT aplicado à saúde digital: captura, processamento, transmissão, visualização e alerta. A simulação no Wokwi permitiu validar a lógica do firmware sem necessidade de hardware físico, enquanto o Node-RED, InfluxDB e Grafana formaram uma infraestrutura realista de monitoramento.

As tecnologias utilizadas - MQTT, Edge Computing com buffer FIFO, séries temporais e dashboards interativos - são amplamente aplicadas em soluções reais de saúde digital, evidenciando a aderência do projeto ao estado da arte do mercado de IoT médico.

A separação clara entre as camadas Edge, Fog e Cloud não apenas atende aos requisitos acadêmicos, mas reflete uma arquitetura robusta, escalável e aplicável em projetos reais de monitoramento cardiológico contínuo, como wearables hospitalares e sistemas de telemetria remota.

Referências

BANKS, A.; GUPTA, R. MQTT Version 3.1.1. OASIS Standard, 2014. Disponível em: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>. Acesso em: mai. 2026.

ESPRESSIF SYSTEMS. ESP32 Technical Reference Manual. Versão 5.1. Shanghai: Espressif, 2023. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. Acesso em: mai. 2026.

INFLUXDATA. InfluxDB Documentation – Time Series Database. 2024. Disponível em: <https://docs.influxdata.com>. Acesso em: mai. 2026.

GRAFANA LABS. Grafana Documentation – Open Source Analytics & Monitoring. 2024. Disponível em: <https://grafana.com/docs>. Acesso em: mai. 2026.

NODE-RED. Node-RED Documentation – Flow-based Programming for the Internet of Things. 2024. Disponível em: <https://nodered.org/docs>. Acesso em: mai. 2026.

ECLIPSE FOUNDATION. Eclipse Mosquitto – An Open Source MQTT Broker. 2024. Disponível em: <https://mosquitto.org>. Acesso em: mai. 2026.

BENETTI, G. et al. IoT-based Patient Monitoring Systems: A Systematic Review. Journal of Medical Internet Research, v. 25, n. 1, 2023.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União, Brasília, 2018.

WOKWI. Wokwi – Online ESP32 and Arduino Simulator. 2024. Disponível em: <https://wokwi.com>. Acesso em: mai. 2026.

Repositório GitHub: https://github.com/ijoaozz/CardioAI_FASE_3

O código-fonte completo, diagramas e prints do dashboard estão disponíveis no repositório do projeto, organizado nas pastas *Parte_1/* e *Parte_2/*.

Relatório elaborado para a Fase 3 do Projeto CardiolA – FIAP, 2026.